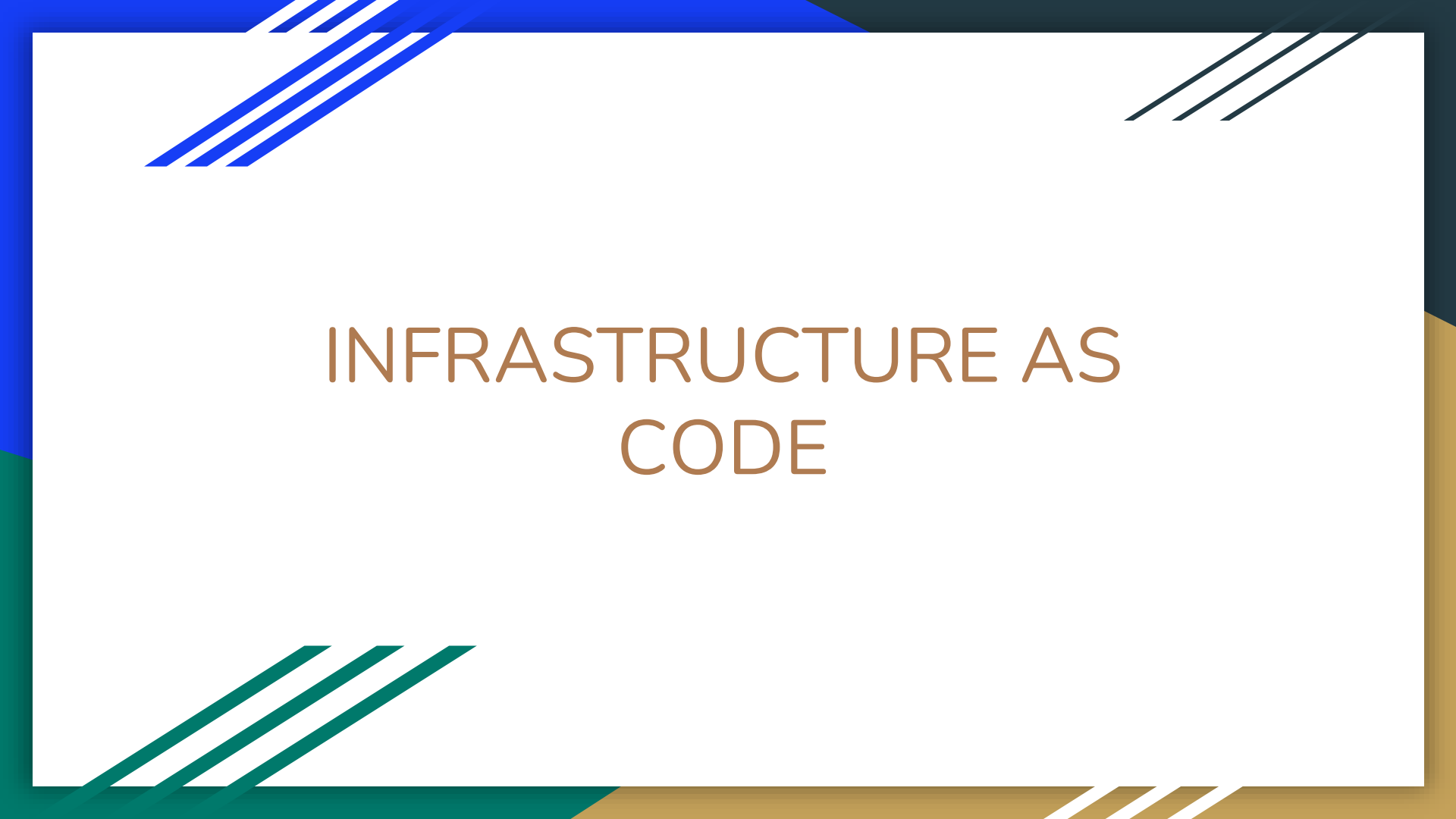




INFRASTRUCTURE AS CODE

UNATTENDED AUTOMATION

Tools run on a schedule to apply, re-apply, and update configuration

BENEFITS OF UNATTENDED:

- **Discover** problems quickly
- Force yourself to **fix** those problems
- Force yourself to **improve** your tools and processes
- Discourages “out of band” changes

AUTOMATE SERVER UPDATES

Automation isn't just for new servers!

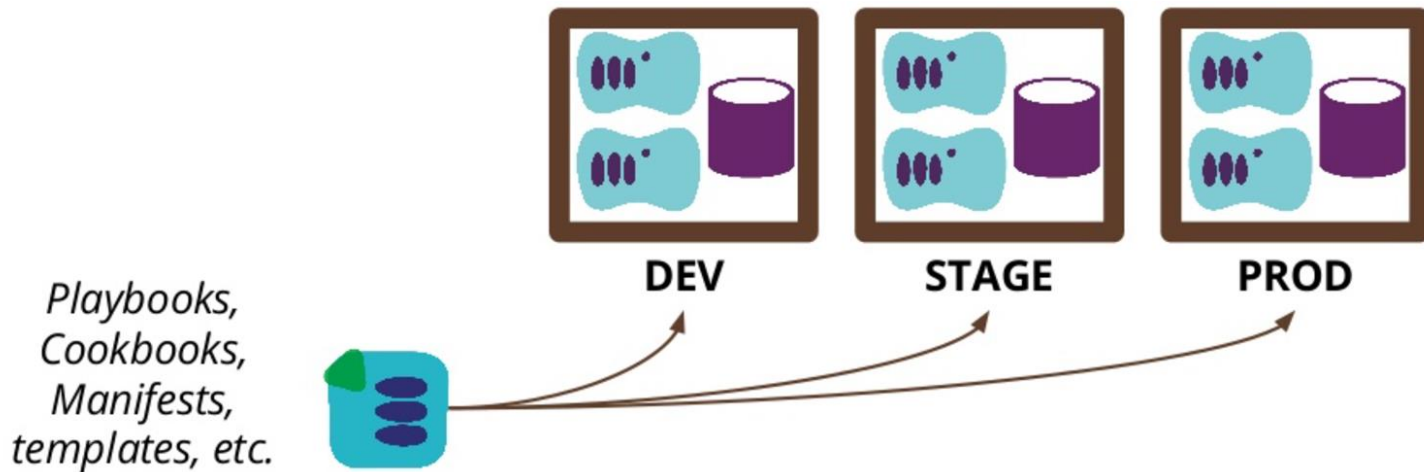
Configuration synchronization Run Chef, Puppet, Ansible, etc. on a schedule

Immutable servers Apply changes by rebuilding servers

Containerized servers Apply changes by deploying new container instances

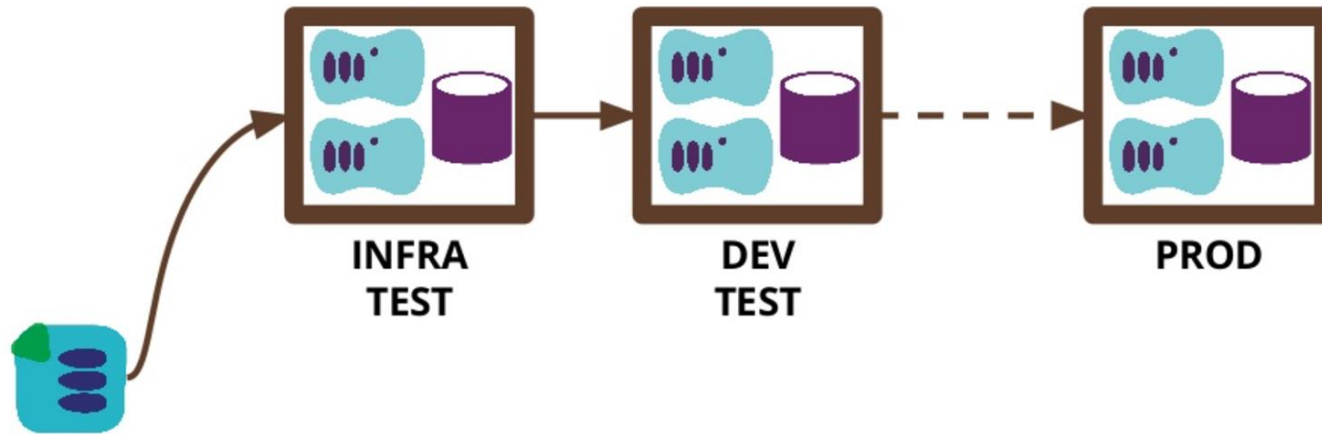
RE-USE & PROMOTE DEFINITIONS

Re-use the same definition files across environments for a given application or service



TEST INFRASTRUCTURE CHANGES

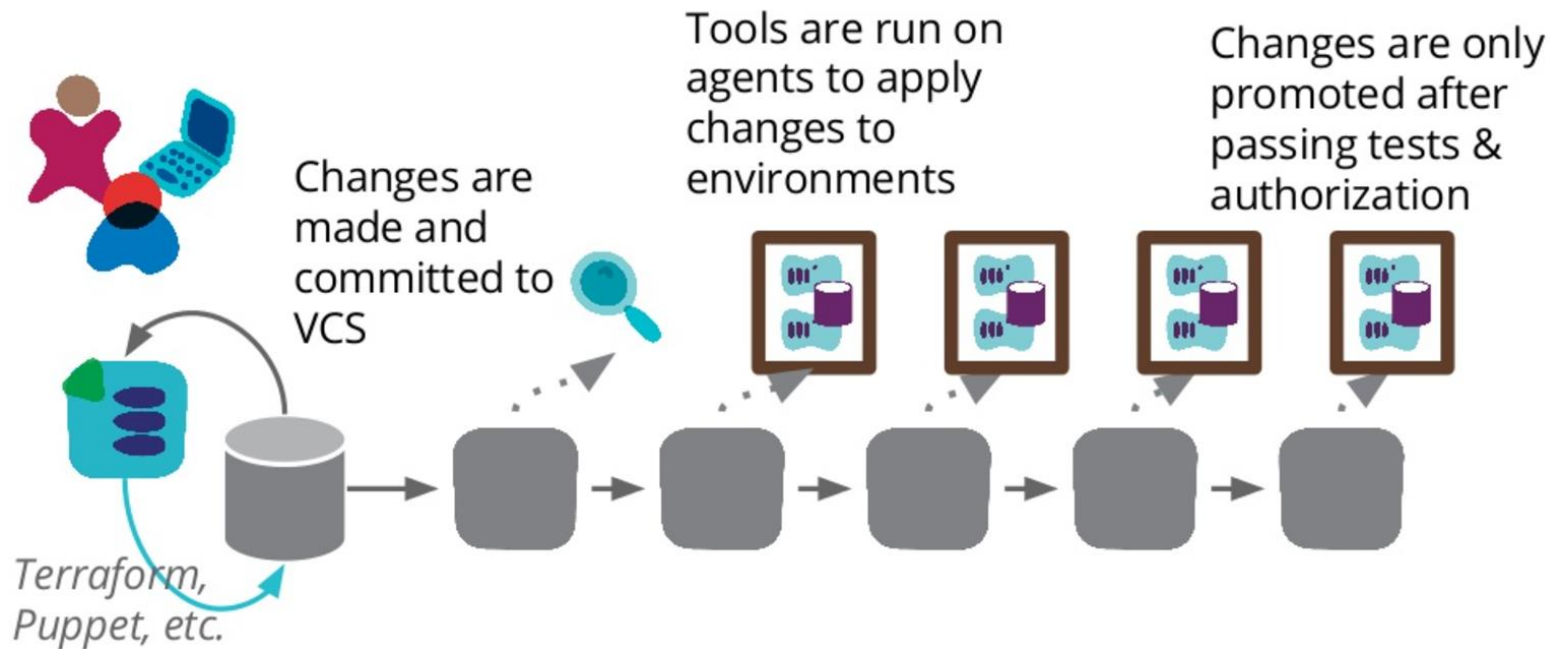
Preventing DevOps



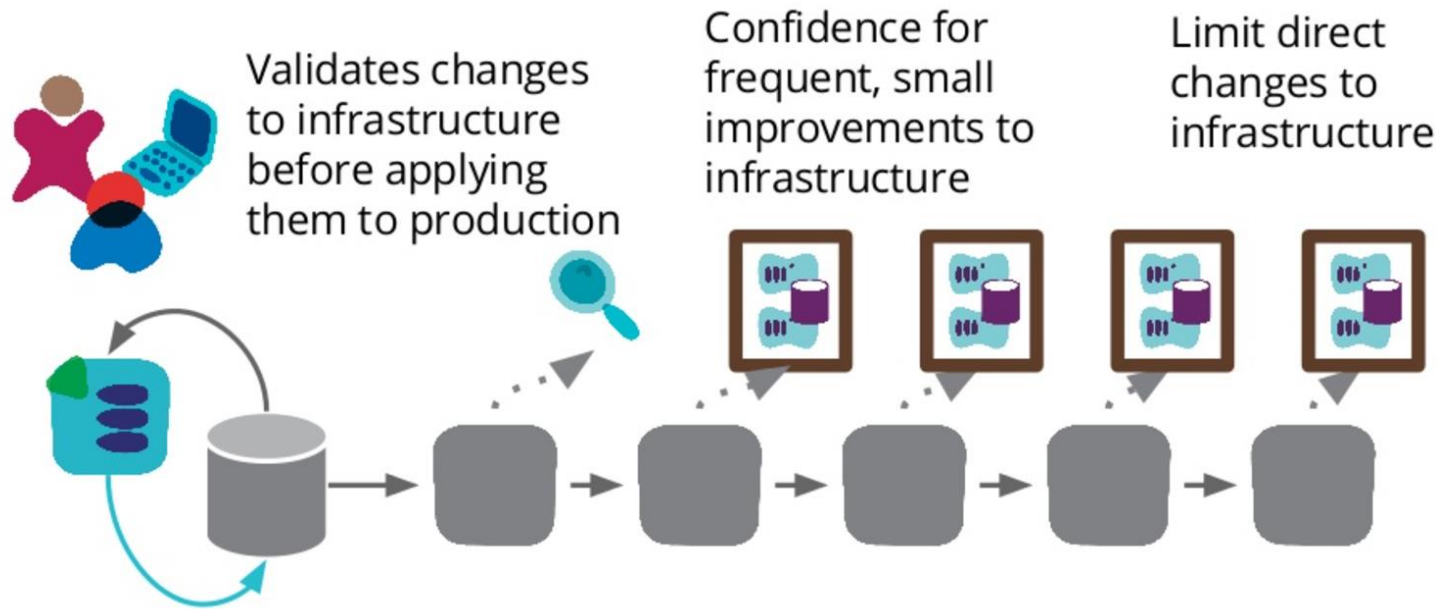


Pipelines

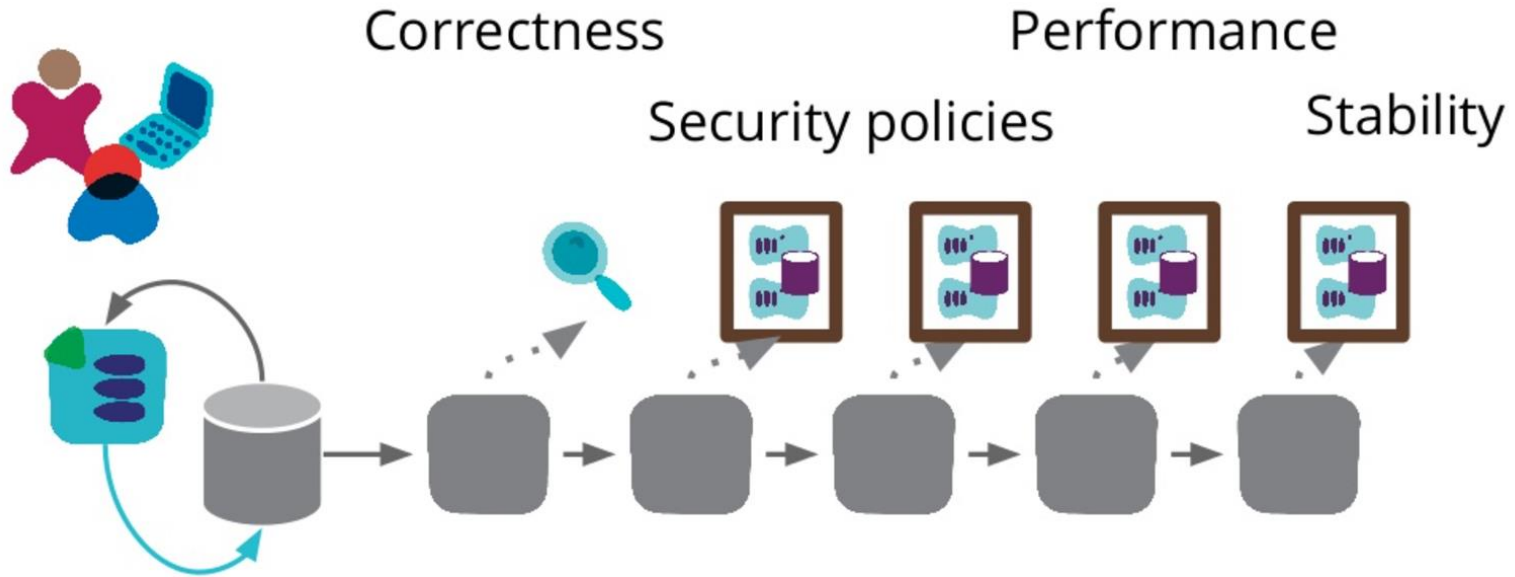
WHAT?



WHY?

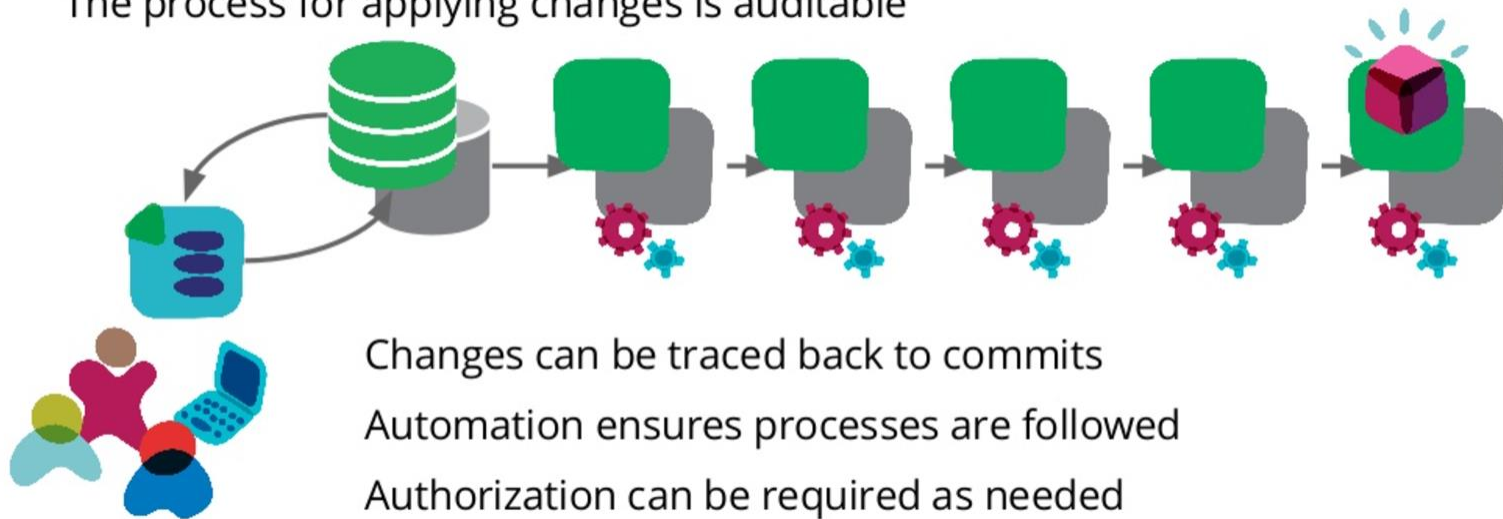


TESTING



GOVERNANCE

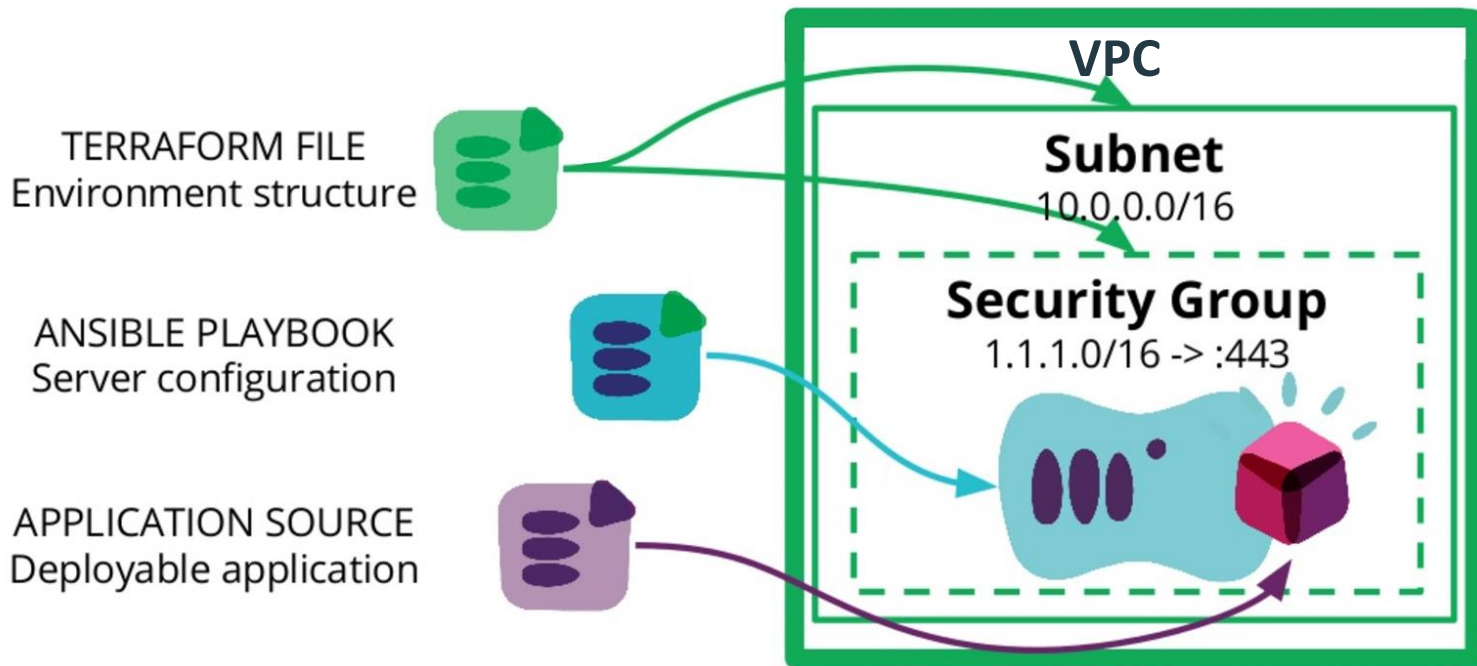
The process for applying changes is auditable





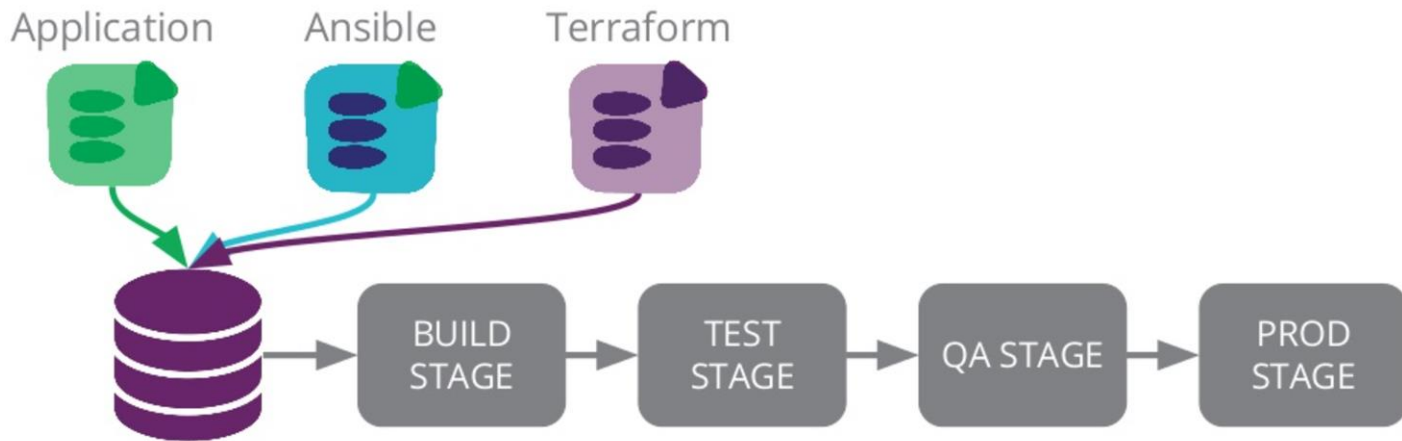
Simple environment example

DEFINING A SIMPLE ENVIRONMENT



SIMPLE PIPELINE DESIGN

Deploy application, configuration, and infrastructure





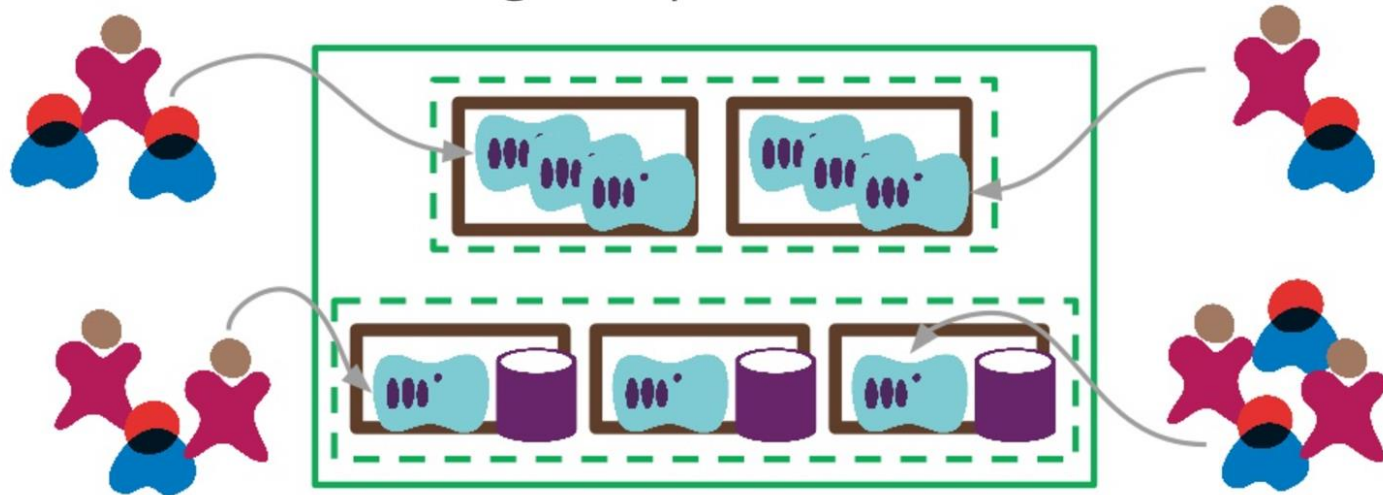
Scaling

ALIGN INFRASTRUCTURE DESIGN TO TEAMS

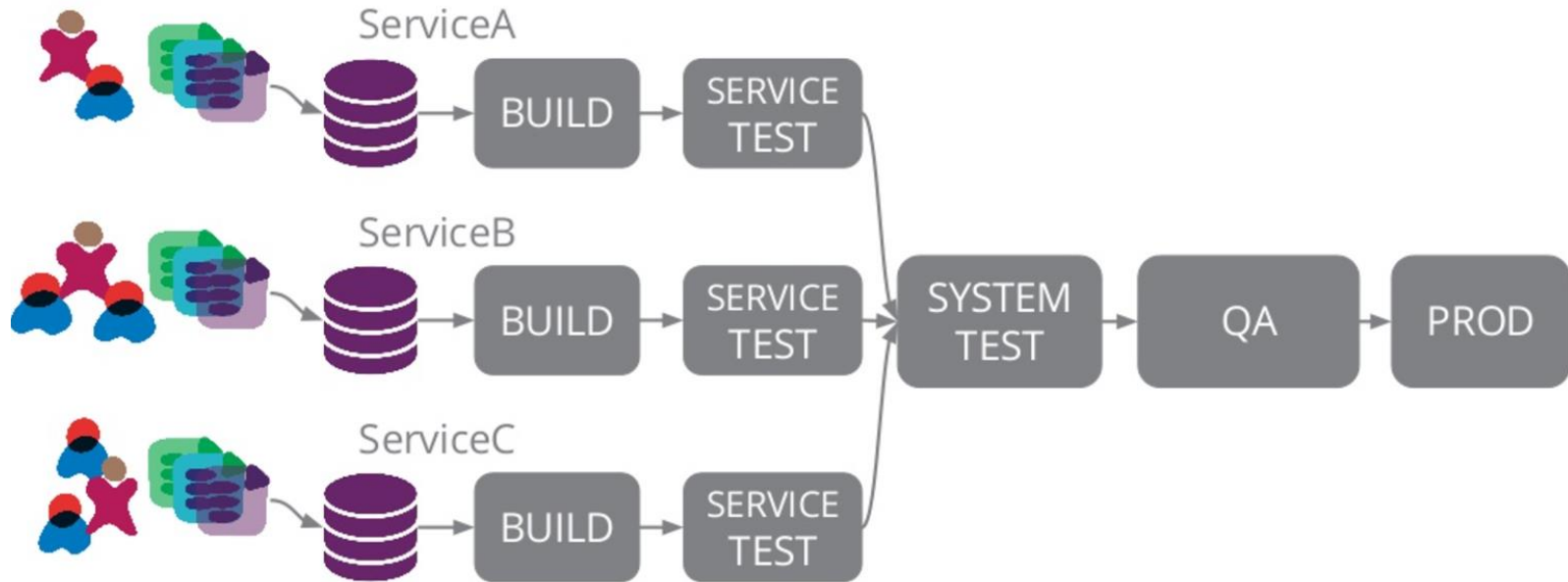
Ensure teams can make the changes they need easily and safely

COMPLEX ENVIRONMENTS

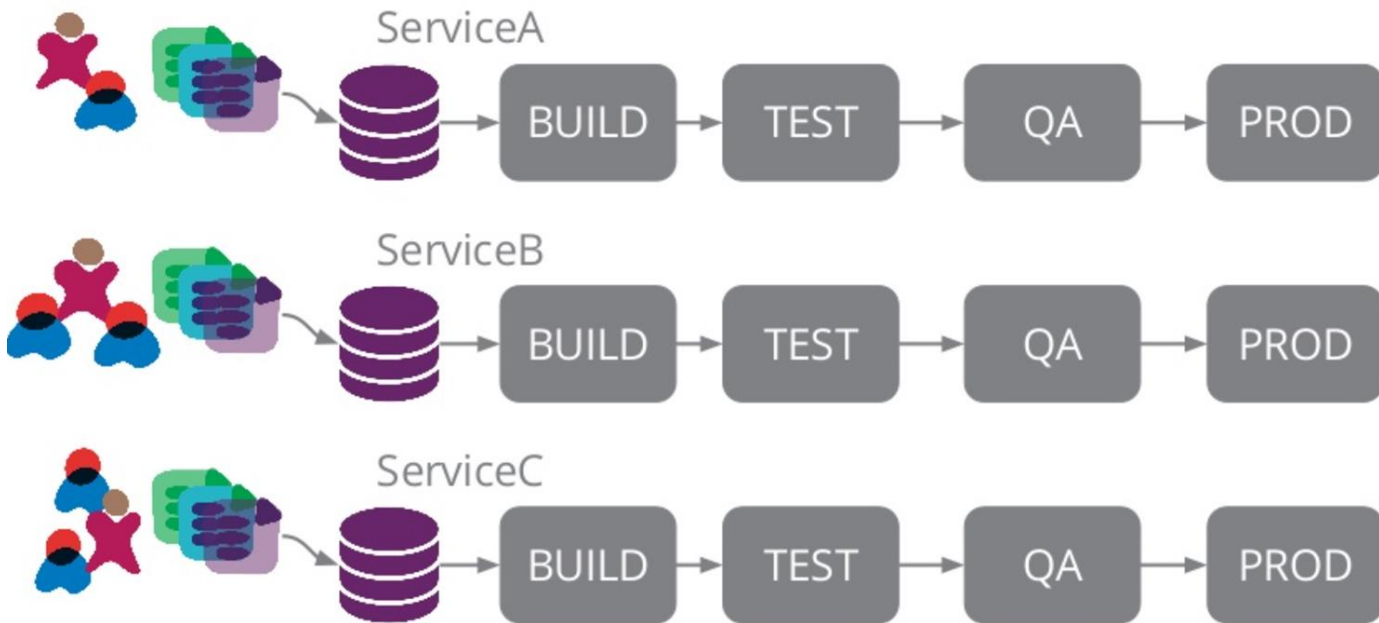
Infrastructure involving multiple teams



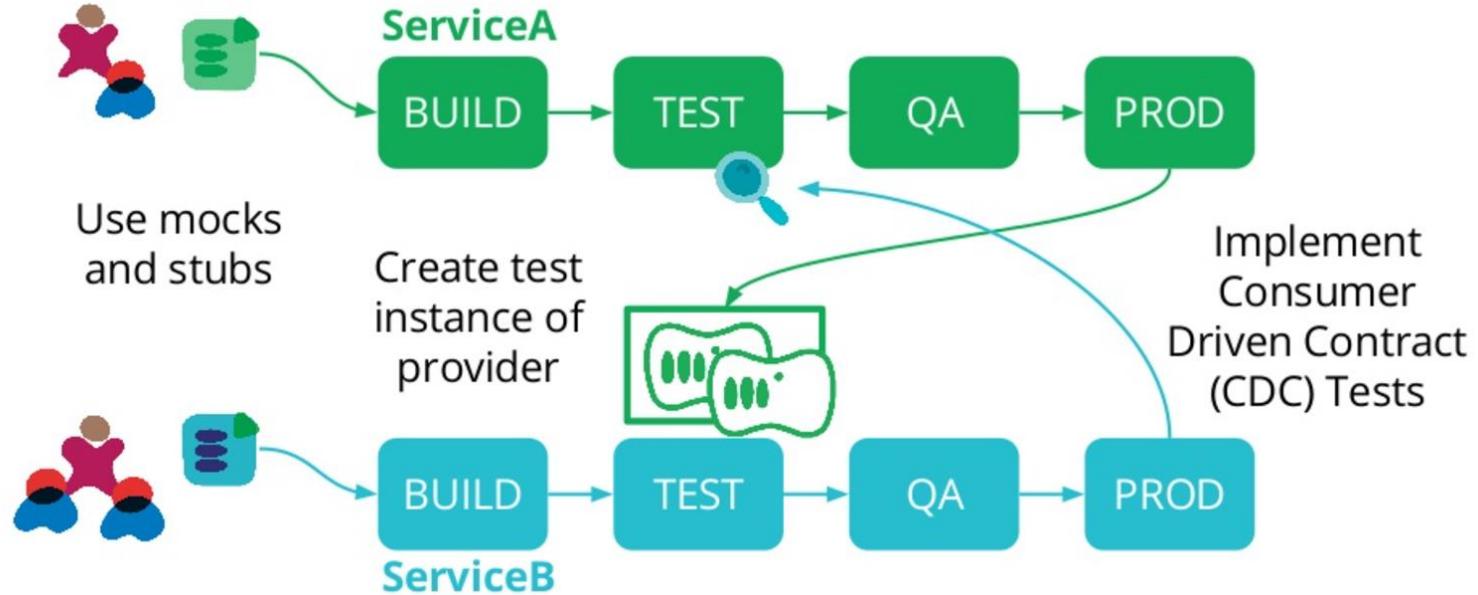
FAN-IN PIPELINE



DECOUPLED PIPELINES

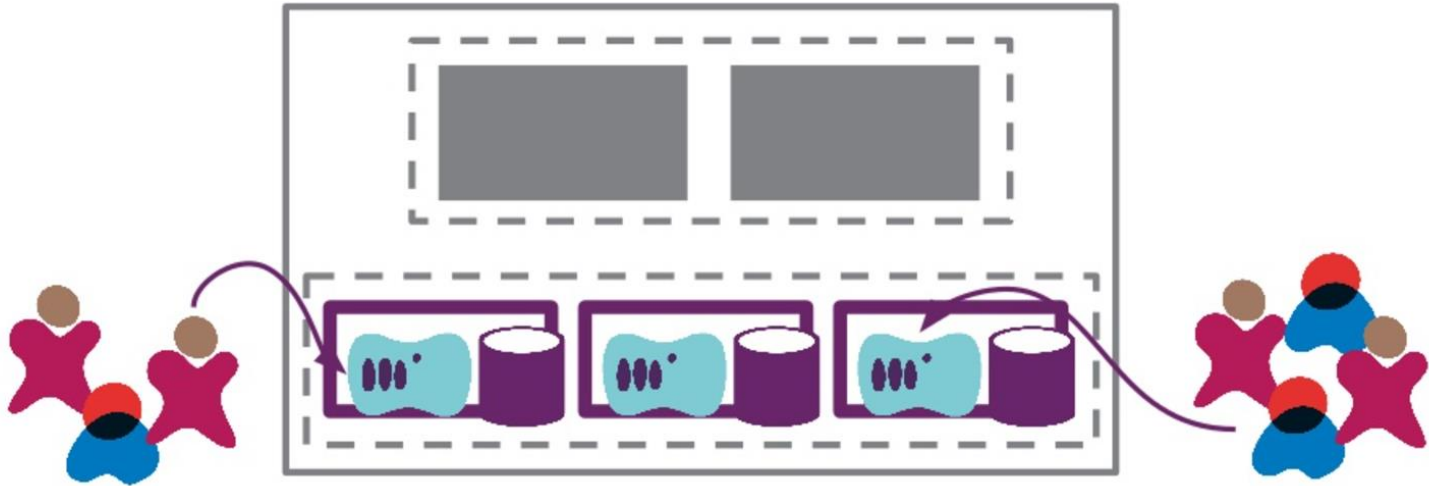


DEPENDENCIES



ISSUE: DUPLICATION

Multiple teams using similar systems, e.g. database clusters



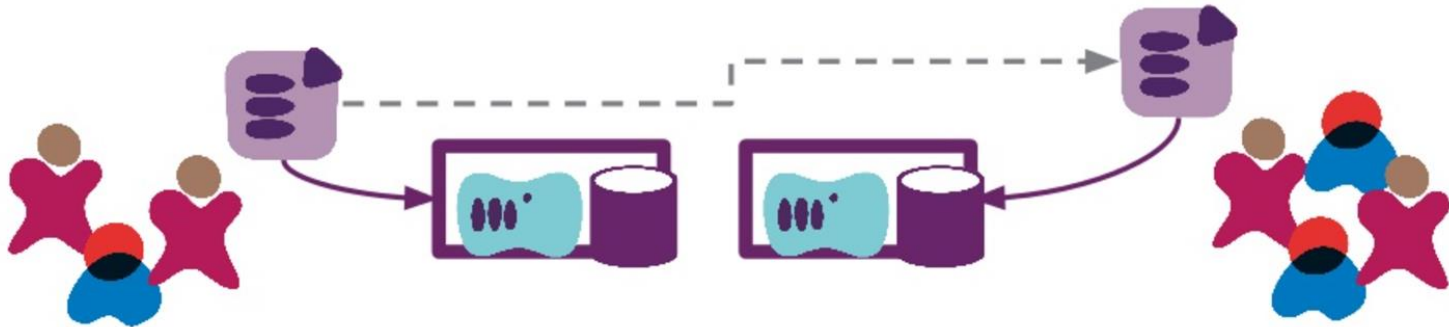
RE-USE BY FORKING DEFINITIONS

Disadvantages:

- Divergence and Inconsistency

Advantages:

- Avoid tight coupling
- Handles diverse requirements



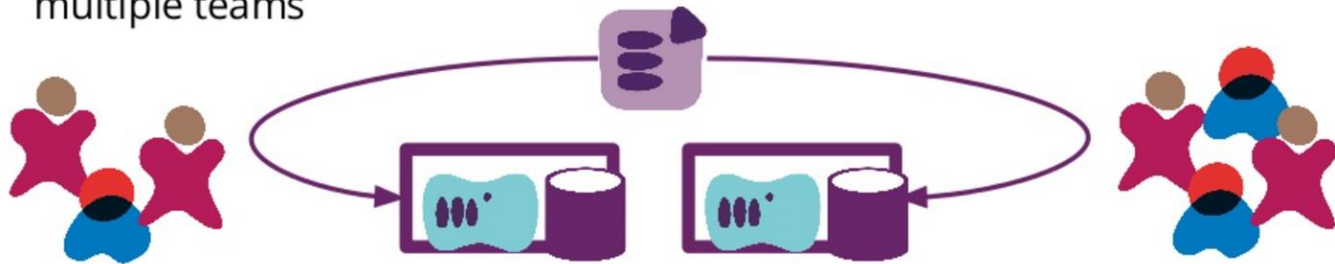
RE-USE WITH DEFINITION LIBRARIES

Challenges:

- Avoid tight coupling, so teams aren't blocked when making changes
- Ownership of code shared by multiple teams

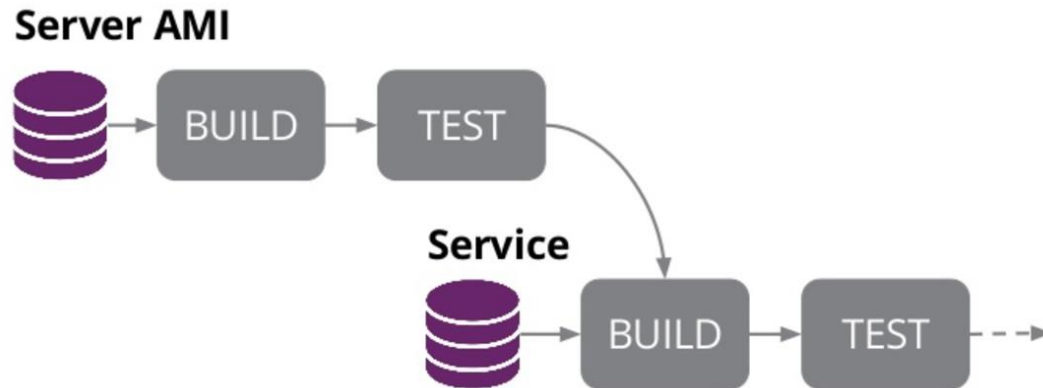
Guidance:

- Use separate pipelines for each
- Use CDC & other dependency testing strategies

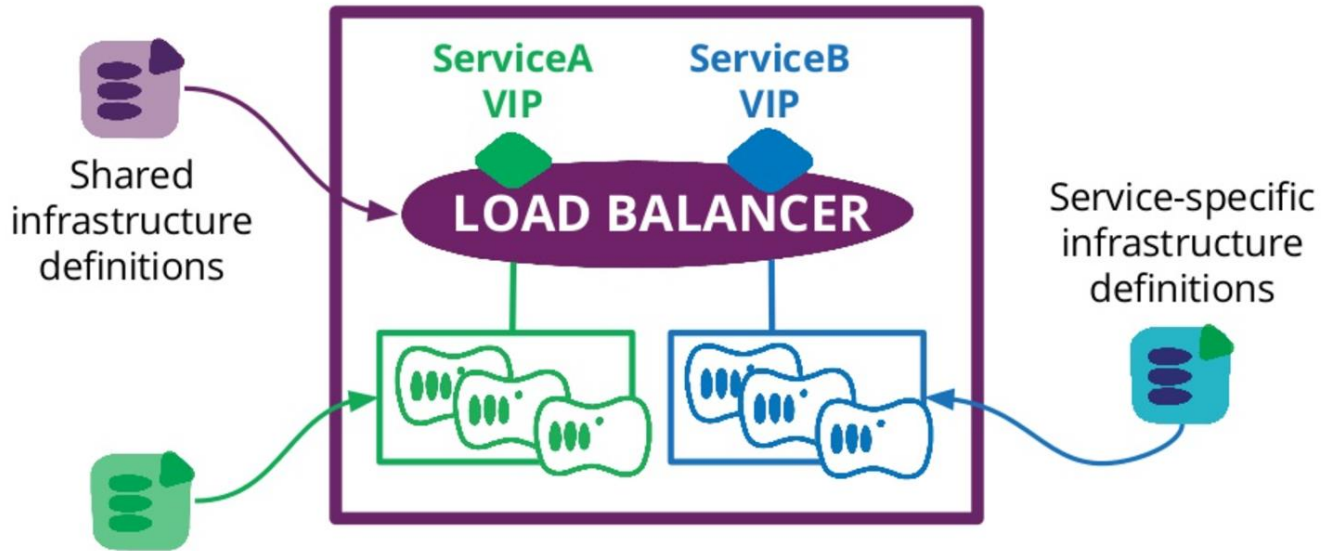


LIBRARY PIPELINE

Test shared definitions before pulling them into dependent pipelines

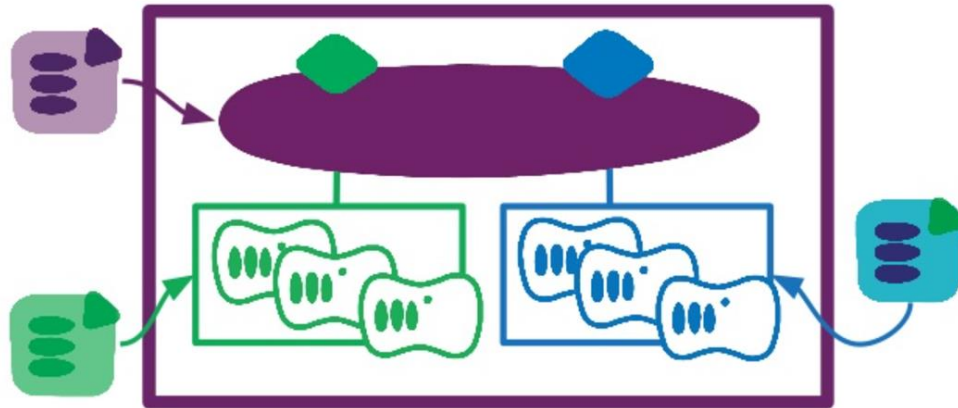


ISSUE: SHARED ELEMENTS



SHARING ELEMENTS

Avoid monoliths - optimize to simplify making changes





Infrastructure as Code (IaC) with Terraform and Ansible

Infrastructure as Code

Describe all infrastructure components by text

- Virtual Machines
- Networks
- Firewalls, Load Balancers ...
- In the end: Complete services ...

Infrastructure as Code

- Use tools to build and configure actual infrastructure on top of an IaaS
- e.g. Terraform, Salt Cloud, Ansible, Openstack Heat

Benefits:

- Reproducible Infrastructure
- Versioned Infrastructure
- Validated Infrastructure
- Documented Infrastructure
- Infrastructure changes become visible
- Re-usable infrastructure

Infrastructure as Code

Short:

- Treat infrastructure as data (text) and use the same techniques as software developers:
- VCS: keep infrastructure definitions versioned
- TDD: test infrastructure using serverspec, testinfra
- CI / CD: infrastructure changes walk through a CI / CD pipeline

Infrastructure as Code

Some challenges:

- How to work securely with API access tokens in a team ?
- Automation Fear
 - a. No confidence in automation tools, because
 - b. servers were not consistent configured, because
 - c. automation is not run frequently ... go to 1

Infrastructure as Code

Virtual machine templates:

- build new infrastructure using pre-built images
- automate building process: e.g. using packer from HashiCorp

Building Infrastructure with Terraform

- DSL (Domain Specific Language) to build infrastructure
- Terraform is a provisioning tool, not a configuration management tool
- Terraform uses a declarative syntax
- Terraform can make incremental changes to existing infrastructure

Building Infrastructure with Terraform

Incremental change needs state information:

- Terraform creates a state file:
terraform.tfstate

How to share this state information between team members ?

- Git, Amazon S3, Openstack Swift, etcd, consul

Resource definition for a virtual machine:

```
38 resource "openstack_compute_instance_v2" "web_1" {
39
40     name = "web_1"
41     flavor_name = "m1.small"
42     key_pair = "sbeffa"
43
44     network {
45         uuid = "${openstack_networking_network_v2.private_ubuntu.id}"
46     }
47
48     user_data = "${template_file.web_1.rendered}"
49     floating_ip = "192.168.122.250"
50     image_id = "e4ce4179-9971-40f5-9c24-1230538ef82e"
51
52 }
```

- Terraform searches for the file main.tf in the current working directory, main entrypoint
- Modularization is supported

Configuring Infrastructure with Ansible

- Ansible is a configuration management tool
- Distinction between configuration management and provisioning is not that easy
- Can also be used to manage infrastructure, e.g. ansible os_server or ec2 modules
- But has no way to store infrastructure state as Terraform does

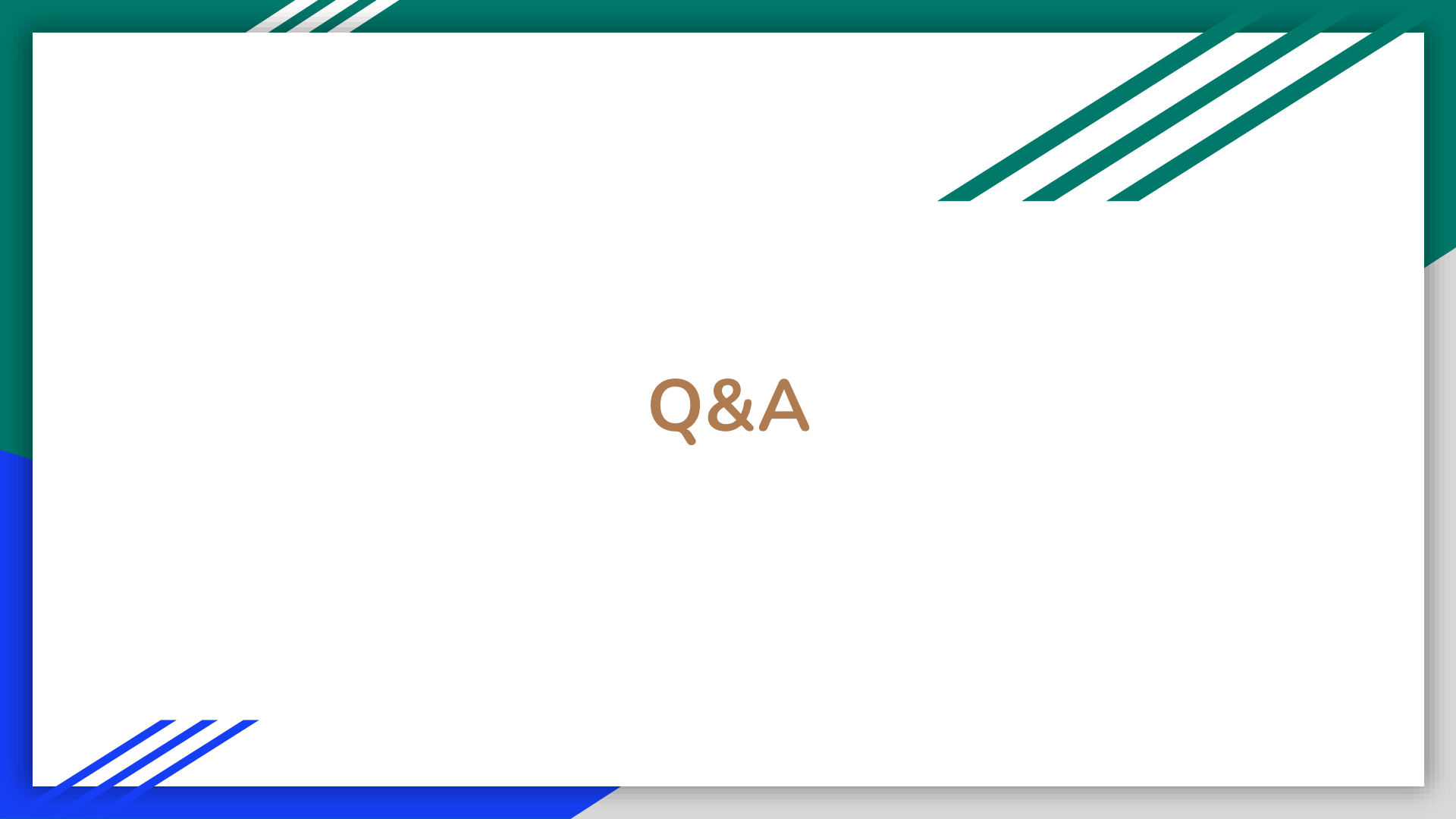
Therefore we use

- Terraform to build (create) infrastructure
- Ansible to configure infrastructure

Configuring Infrastructure with Ansible

How to glue Terraform and Ansible together ?

- Custom orchestrator which first executes Terraform and Ansible afterwards



Q&A